# XML Web Services Assignment

| | |
|---|---|
| **Student Name** | Yash Biware |
| **Roll Number** | EN24CS3T10024 |
| **Assignment** | XML Web Services Implementation |
| **Subject** | Web Services & SOA |
| **GitHub** | https://github.com/YashBiware/weather-api |
| **Live URL** | https://weather-api-p3wd.onrender.com |

## Project Overview

This assignment implements a secured XML-based REST API using Python (FastAPI) that demonstrates real service-to-service integration. The API consumes the Open-Meteo external weather service, uses JWT (JSON Web Token) for authentication, and exchanges data in XML format. The service is deployed live on Render and can be tested using Postman.

### Technology Stack

| Component | Technology | Purpose |
|---|---|---|
| Backend Framework | Python + FastAPI | REST API server |
| Data Format | XML | Request & Response format |
| Authentication | JWT (Bearer Token) | Secure endpoint access |
| External Service | Open-Meteo API | Live weather data |
| Deployment | Render (Free Tier) | Cloud hosting |
| Testing Tool | Postman | API testing & demonstration |
| Version Control | GitHub | Source code repository |

## API Endpoints Reference

All endpoints exchange data in XML format. Protected endpoints require a Bearer token in the Authorization header obtained after registration or login.

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| POST | /auth/register | No | Register new user, returns JWT token |
| POST | /auth/login | No | Login existing user, returns JWT token |
| GET | /weather/{city} | Yes | Get live weather data for any city |
| GET | /weather/history/mine | Yes | View your weather query history |

---

## Section 1: Service Deployment on Render

The weather API is deployed as a live web service on Render (render.com). Render automatically pulls the code from GitHub and hosts it on a public URL, making the API accessible 24/7 from anywhere on the internet.

### Step 1.1 — Render Projects Dashboard

After signing in to render.com with your GitHub account, the Projects dashboard shows all your deployed services. The weather-api service is organized under 'My Project' in the Production environment.

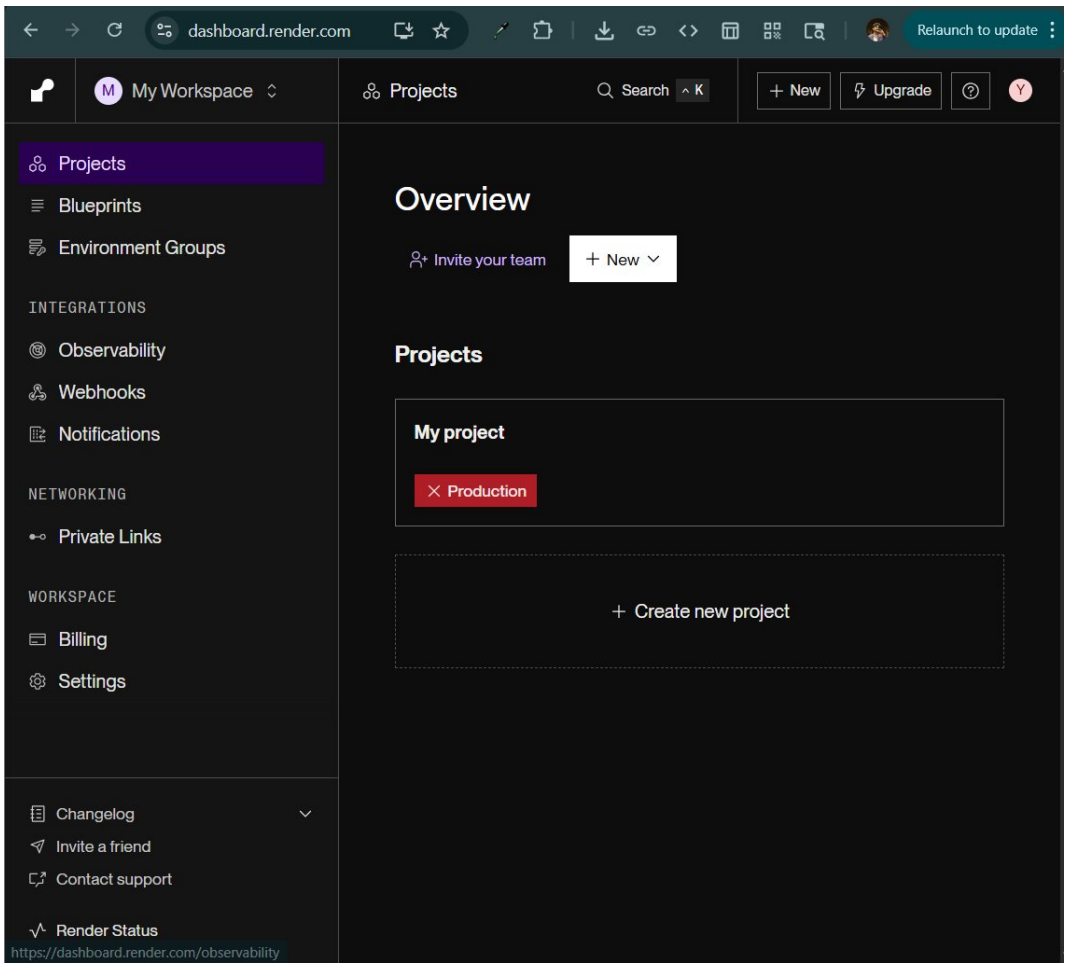


*Figure 1: Render Projects Dashboard showing 'My project' with Production environment*

## Step 1.2 — Service Settings (Repository & Branch)

Inside the weather-api service, the Settings tab shows the GitHub repository linked for auto-deployment. Any push to the 'main' branch triggers an automatic re-deploy on Render.

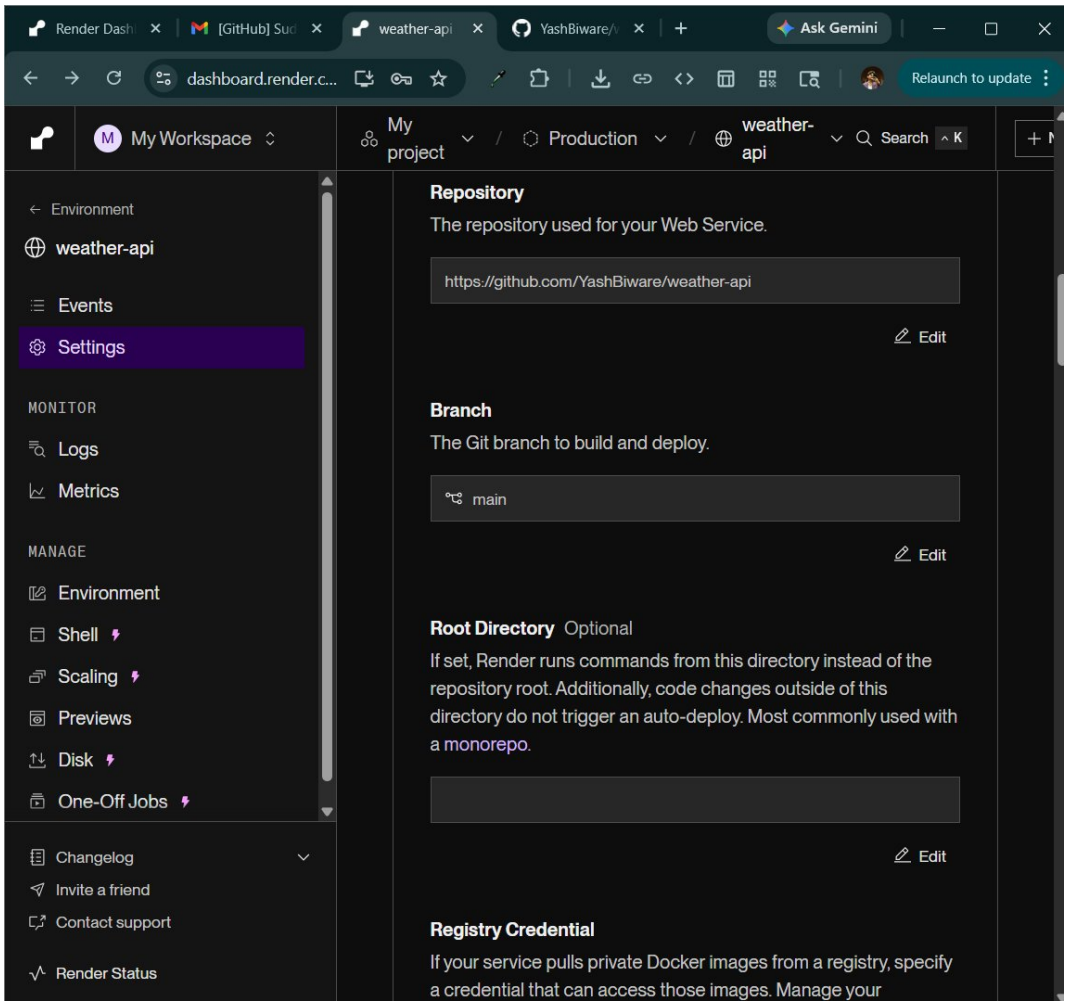


*Figure 2: Service Settings showing GitHub repository (YashBiware/weather-api) and branch (main)*

## Step 1.3 — Service Overview & Live URL

The service Events page shows the deployment history. The green checkmarks confirm successful deployments. The live URL is displayed at the top of the page:

**Live URL:** https://weather-api-p3wd.onrender.com

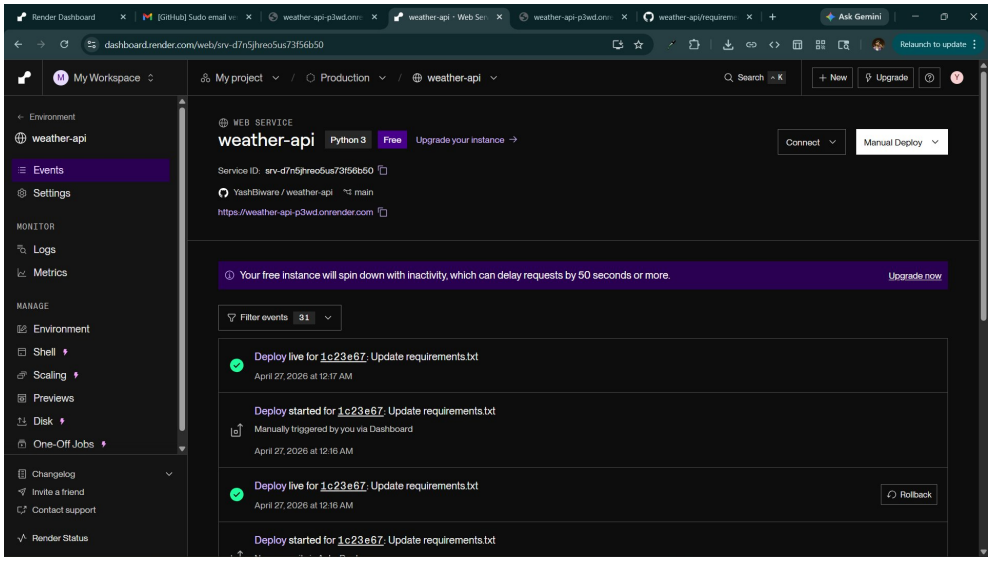


*Figure 3: Render Service Events page — deployment history with successful green checkmarks*

## Step 1.4 — Render Project Production Environment

The production environment shows weather-api as 'Deployed' with Python 3 runtime, confirming the service is live and running in the cloud.

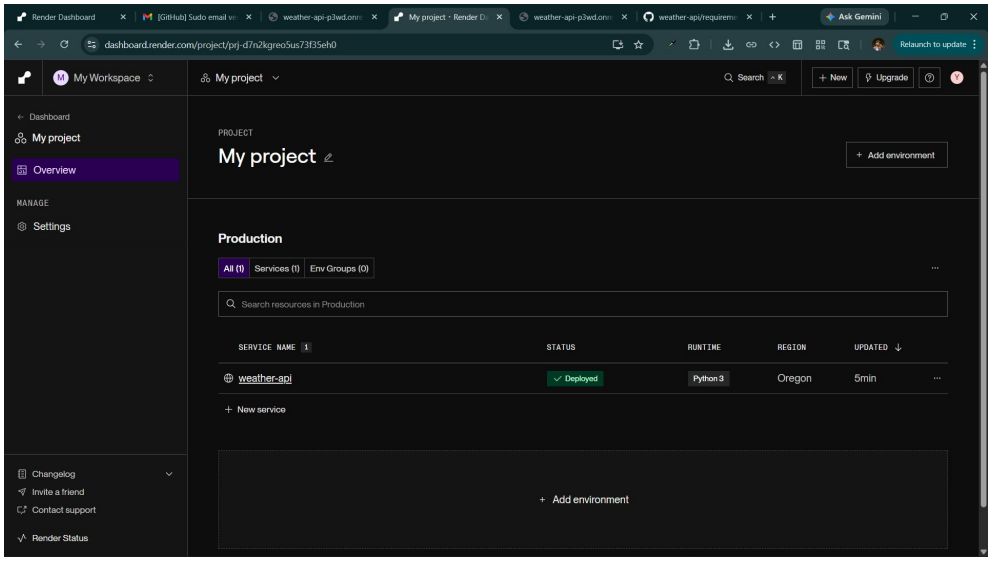


*Figure 4: Production environment showing weather-api as Deployed (Python 3, Oregon region)*

## Section 2: Live Service Verification in Browser

The root endpoint of the API returns an XML response confirming the service is running. Open the URL directly in a browser to verify the API is live.

### Step 2.1 — Root Endpoint Check

Open the following URL in your browser:

```
https://weather-api-p3wd.onrender.com
```

The browser displays an XML document confirming the API is active. This proves the service is live and accessible publicly.

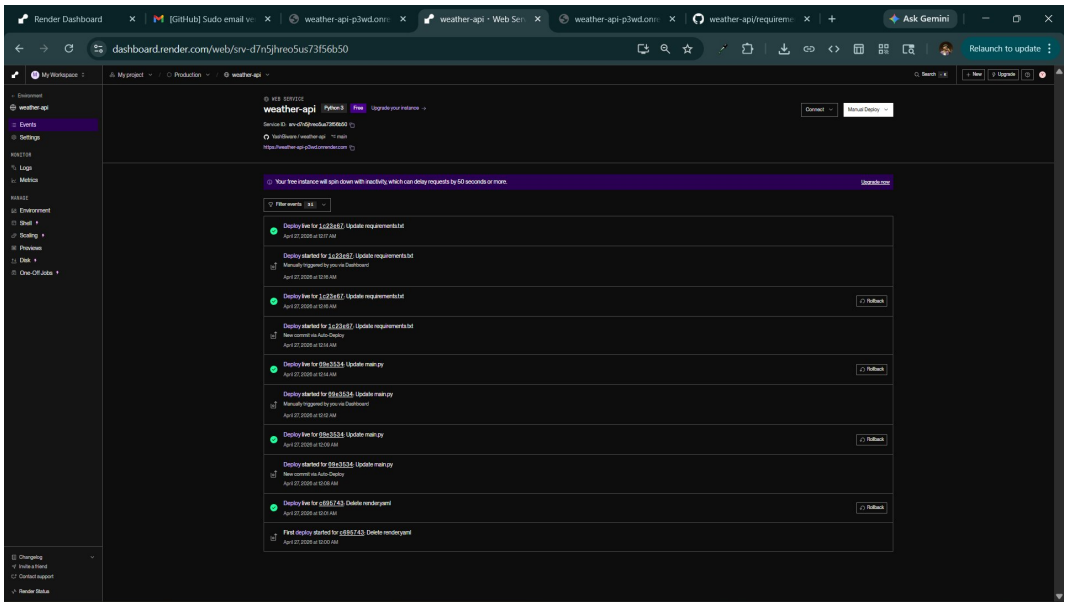


*Figure 5: Browser showing XML response — 'Weather XML API is running' confirming live deployment*

## Section 3: Testing the API Using Postman

Postman is used to send HTTP requests to the API and view XML responses. All four core operations are demonstrated below. Follow these steps in order.

### Step 3.1 — Register a New User (POST /auth/register)

Instructions:

1. Open Postman and create a new request tab.
2. Set Method to **POST**.
3. Enter URL:

```
https://weather-api-p3wd.onrender.com/auth/register
```

4. Click **Body** tab → select **raw** → change dropdown from Text to **XML**.
5. Paste the following XML body:

```
<?xml version="1.0" encoding="UTF-8"?>
<credentials>
  <username>yash</username>
  <password>1234</password>
</credentials>
```

6. Click **Send**. The response will contain a JWT token — **copy this token**.

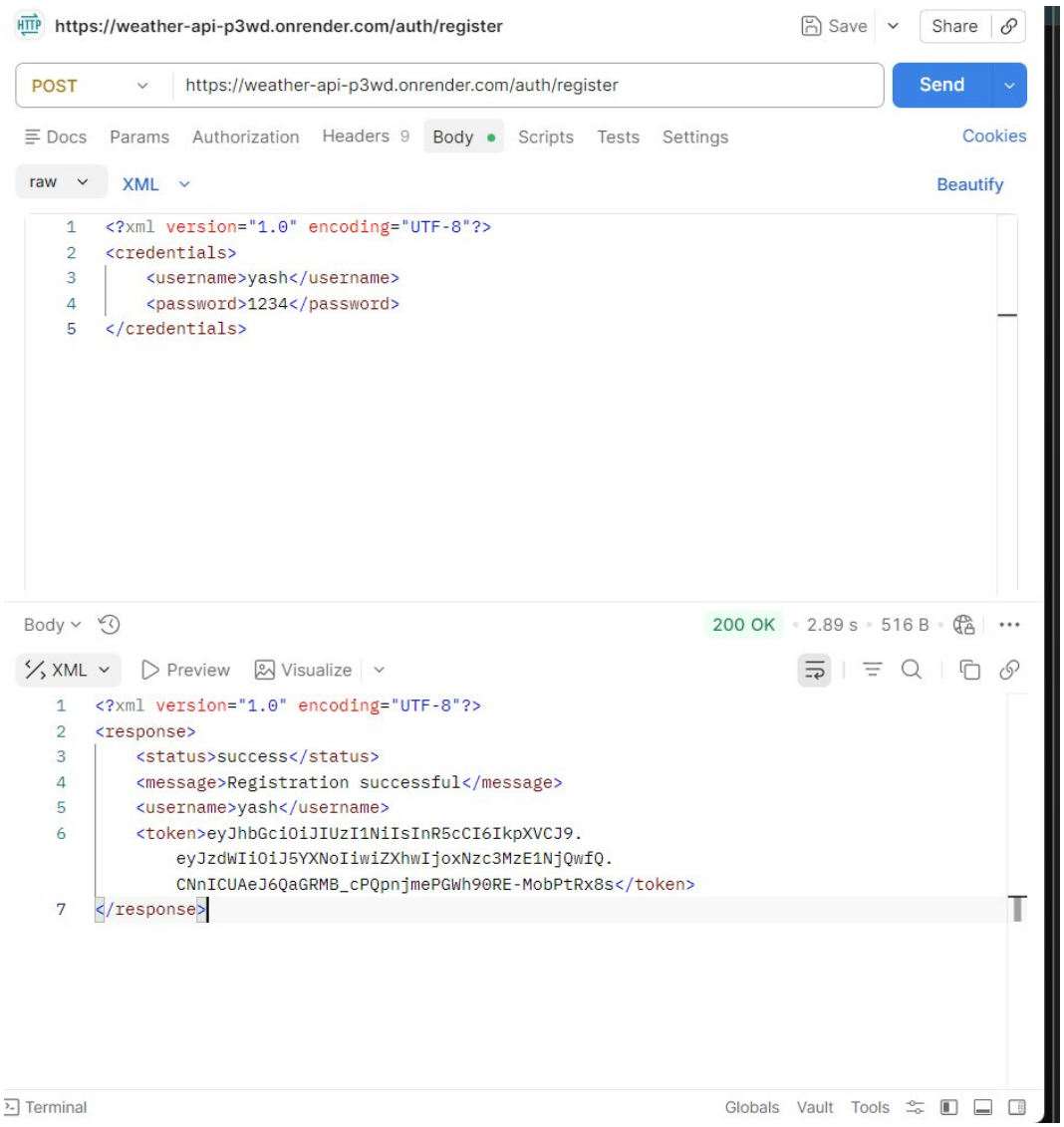


*Figure 6: POST /auth/register — 200 OK response with JWT token in XML format. The token is used to authenticate all subsequent requests.*

## Step 3.2 — Get Live Weather Data (GET /weather/Indore)

Instructions:

1. Create a new request tab in Postman.

2. Set Method to **GET**.

3. Enter URL:

```
https://weather-api-p3wd.onrender.com/weather/Indore
```

4. Click the **Authorization** tab → Auth Type → select **Bearer Token**.

5. Paste the JWT token copied from the Register step into the Token field.

6. Click **Send**.

This endpoint calls the Open-Meteo geocoding and forecast APIs internally, retrieving real-time weather data for Indore and returning it as XML.

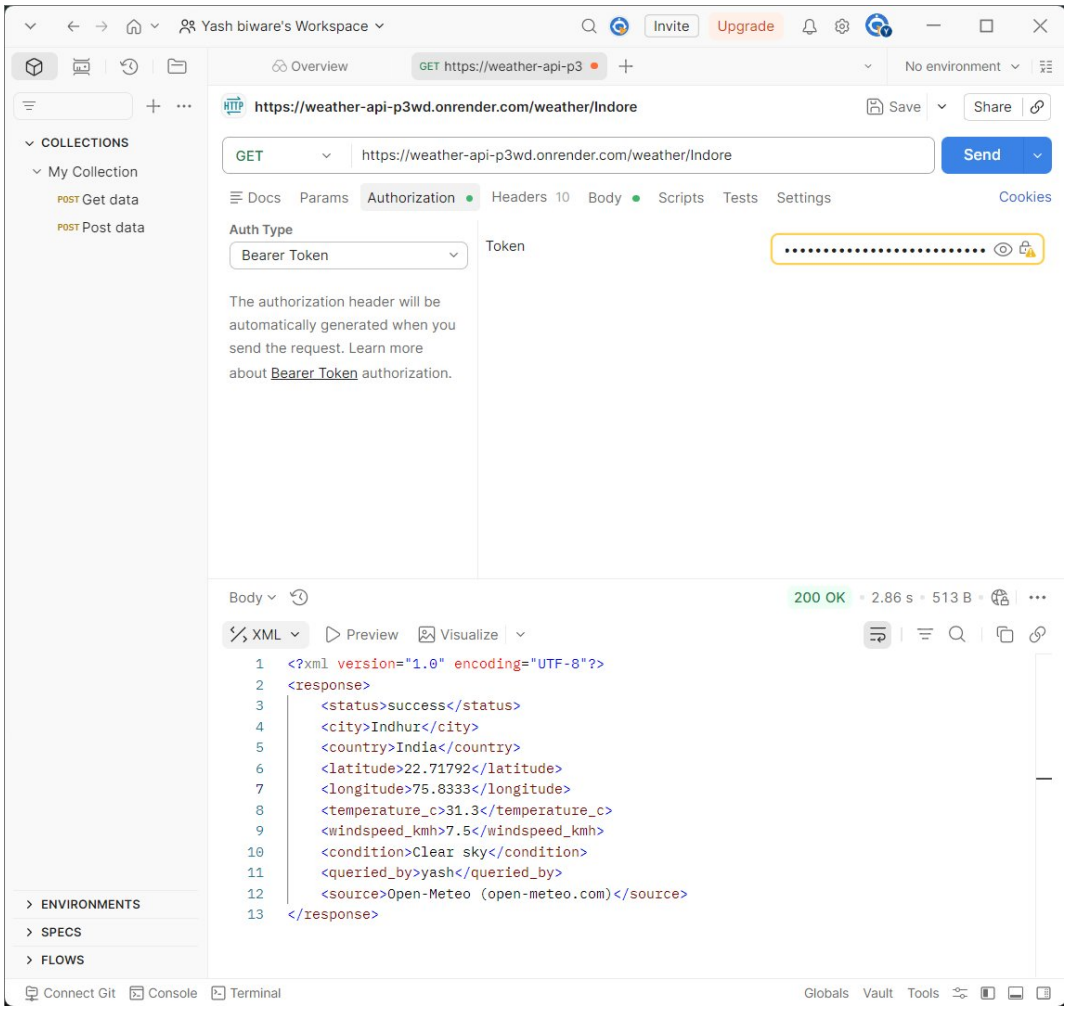


*Figure 7: GET /weather/Indore — 200 OK with live XML weather data. Shows temperature (31.3°C), wind speed (7.5 km/h), condition (Clear sky), and source (Open-Meteo).*

## Step 3.3 — View Query History (GET /weather/history/mine)

Instructions:

1. Create a new request tab in Postman.
2. Set Method to **GET**.
3. Enter URL:

```
https://weather-api-p3wd.onrender.com/weather/history/mine
```

4. Click the **Authorization** tab → Auth Type → **Bearer Token** → paste your token.
5. Click **Send**.

This endpoint returns all weather queries made by the logged-in user, stored in-memory during the current server session.

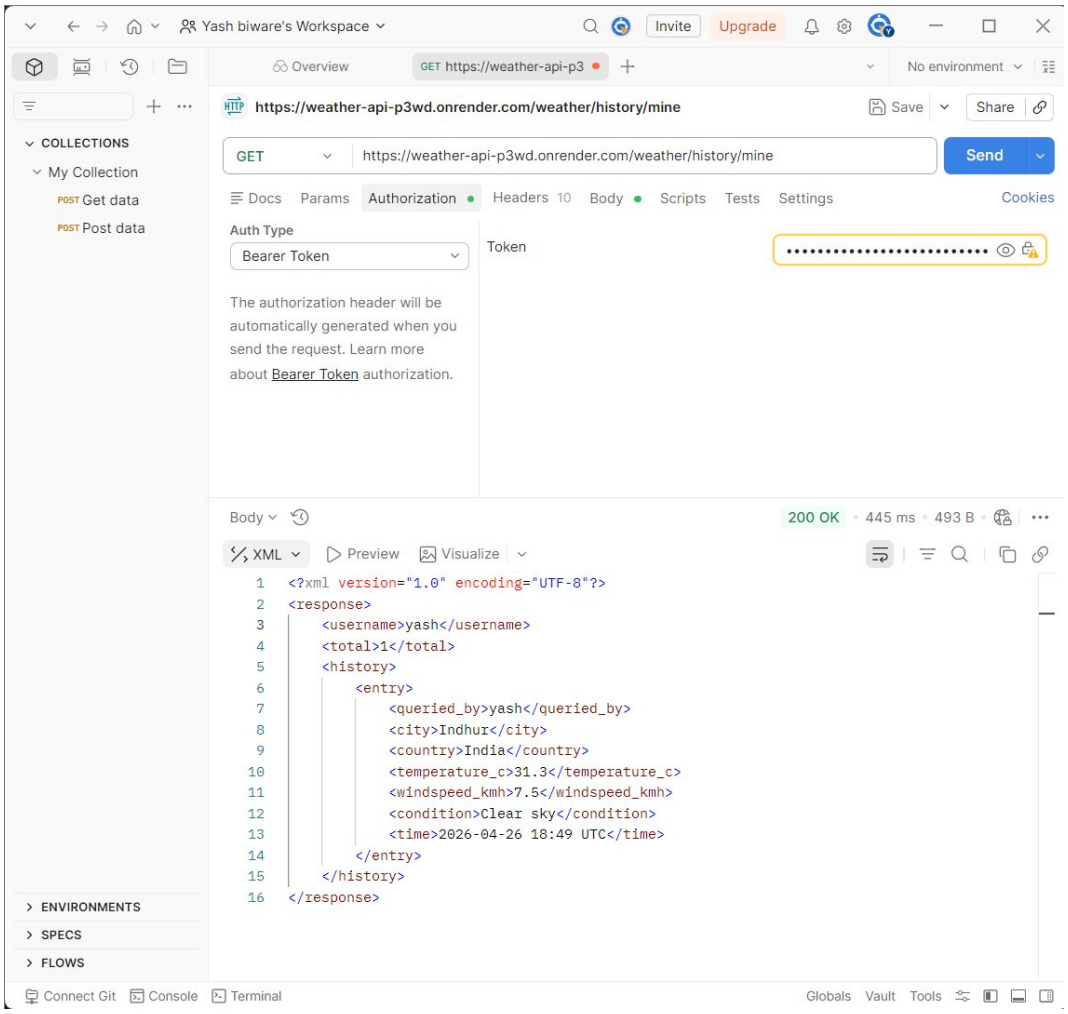


*Figure 8: GET /weather/history/mine — 200 OK with XML history showing 1 entry (Indore, 31.3°C, Clear sky, queried at 2026-04-26 18:49 UTC).*

## Section 4: GitHub Repository

The source code is version-controlled and publicly hosted on GitHub. Render automatically deploys from this repository on every push to main.

### Repository Details

| | |
|---|---|
| **Repository URL** | https://github.com/YashBiware/weather-api |
| **Branch** | main |
| **Files** | main.py, requirements.txt, README.md |
| **Runtime** | Python 3.14 (Render managed) |

### How to Access and Run Locally

Step 1 — Clone the repository:

```
git clone https://github.com/YashBiware/weather-api.git
cd weather-api
```

Step 2 — Install dependencies:

```
pip install -r requirements.txt
```

Step 3 — Start the server:

```
python -m uvicorn main:app --reload
```

Step 4 — Test locally (replace the Render URL with localhost):

```
POST http://localhost:8000/auth/register
GET http://localhost:8000/weather/Indore
GET http://localhost:8000/weather/history/mine
```

## Section 5: Service Architecture & How It Works

### Authentication Flow

1. Client sends XML credentials (username + password) to /auth/register or /auth/login.
2. Server hashes the password using bcrypt and stores in memory.
3. Server generates a JWT token (expires in 30 minutes) signed with a secret key.
4. Client includes the token in every subsequent request as: **Authorization: Bearer <token>**
5. Server validates the token on each protected endpoint call.

### External Service Integration (Open-Meteo)

When a client calls GET /weather/{city}, the server performs a two-step chain of external API calls — this is the core service-to-service integration:

**Step A:** Call Open-Meteo Geocoding API to convert city name → latitude/longitude
URL: https://geocoding-api.open-meteo.com/v1/search?name={city}

**Step B:** Call Open-Meteo Forecast API with the coordinates
URL: https://api.open-meteo.com/v1/forecast?latitude=...&longitude=...&current_weather=true

**Step C:** Package the weather data as an XML response and return to the client.

### XML Request & Response Examples

**Register Request (XML body sent by client):**

```
<?xml version="1.0" encoding="UTF-8"?>
<credentials>
  <username>yash</username>
  <password>1234</password>
</credentials>
```

**Register Response (XML returned by server):**

```
<?xml version="1.0" encoding="UTF-8"?>
<response>
  <status>success</status>
  <message>Registration successful</message>
  <username>yash</username>
  <token>eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...</token>
</response>
```

**Weather Response (XML returned by server):**

```
<?xml version="1.0" encoding="UTF-8"?>
<response>
  <status>success</status>
  <city>Indore</city>
  <country>India</country>
  <latitude>22.71792</latitude>
  <longitude>75.8333</longitude>
  <temperature_c>31.3</temperature_c>
  <windspeed_kmh>7.5</windspeed_kmh>
  <condition>Clear sky</condition>
```

```
  <queried_by>yash</queried_by>
  <source>Open-Meteo (open-meteo.com)</source>
</response>
```

## Section 6: Assignment Completion Summary

| Requirement | Implementation | Status |
|---|---|---|
| REST API | FastAPI (Python) with 4 endpoints | ✓ **Complete** |
| XML Format | All requests & responses use XML | ✓ **Complete** |
| Security | JWT Bearer Token authentication | ✓ **Complete** |
| External Web Service | Open-Meteo API (geocoding + weather) | ✓ **Complete** |
| Service-to-Service | Two chained external API calls per request | ✓ **Complete** |
| GitHub Repository | github.com/YashBiware/weather-api | ✓ **Complete** |
| Deployed & Live | https://weather-api-p3wd.onrender.com | ✓ **Complete** |
| Tested via Postman | All 4 endpoints demonstrated with screenshots | ✓ **Complete** |

All assignment requirements have been fully implemented and demonstrated. The API is publicly accessible and can be tested at any time using the live Render URL.